LEVEL 5

Chapter 6: Distributed Systems

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026

Chapter 6: Distributed Systems

← *Chapter 5: Database Deep Dive* · *Index* · *Next* → *Chapter 7: Messaging & Event-Driven Architecture*

This is intentionally the deepest chapter in the roadmap — distributed-systems reasoning is what separates a mid-level HLD answer from a senior/staff one. Take this one slower than the others.

6.1 The Fundamental Problem

Simple explanation: a distributed system is any system made of multiple machines that must coordinate over a network to do their job. The single fact that makes this hard: **the network is unreliable, and you cannot tell the difference between "slow" and "dead."**

If you call a service and get no response, you genuinely cannot know whether: the request never arrived, the request arrived and is still processing, the request arrived and succeeded but the response was lost on the way back, or the server crashed mid-processing. All four look identical from the caller's side — a timeout. This single fact — **you cannot distinguish a slow node from a dead one** — is the root cause of nearly every pattern in this chapter. Every technique below exists to make a system behave sanely *despite* this fundamental uncertainty.

Analogy: you text a friend "on your way?" and get no reply for ten minutes. Did the message fail to send? Did they see it and are typing a reply? Did their phone die? Are they driving and can't check? You genuinely can't tell — and whatever you do next (wait longer, call them, leave without them) is a judgment call under uncertainty, exactly like a timeout/retry policy is.

Partial failure is the specific flavor of this that single-machine programming never prepares you for: in a distributed call chain of 5 services, it's entirely normal for 3 to succeed and 2 to fail — there's no atomic "the whole operation succeeded or failed" unless you deliberately engineer it (which is exactly what Sagas and 2PC, covered later, are for).

6.2 Timeouts, Retries, Backoff, and Jitter

Timeouts: every network call must have a timeout — without one, a single unresponsive dependency can hold a thread/connection open indefinitely, and enough of those exhaust your connection pool and take down an otherwise-healthy service. This is arguably the single most important, most-skipped detail in real systems — "no timeout" is a very common root cause in postmortems.

Retries: if a call fails or times out, try again — but only for **transient** failures (network blip, momentary overload) and *only* for operations that are safe to retry, which brings you straight to idempotency (below). Retrying a non-idempotent "charge the card" call blindly can double-charge a user.

Exponential backoff: instead of retrying immediately (which can pile onto an already-struggling service and make things worse), wait progressively longer between attempts: 100ms, 200ms, 400ms, 800ms... This gives a struggling downstream service room to recover instead of being retry-bombed by every caller simultaneously.

Jitter: add randomness to the backoff delay. Without jitter, if 10,000 clients all failed at the same moment (e.g., a brief service blip), they'll all retry at *exactly* the same computed backoff intervals — synchronized waves of retries that themselves cause the exact overload they're trying to avoid. Jitter (e.g., a random delay between 0 and the computed backoff value — "full jitter") smears those retries out over time.

> **Interview question:** "You added retries with exponential backoff to a payment service call. A downstream outage happens anyway, worse than before. Why?"
> **Ideal senior answer:** "Most likely a retry storm — if I didn't cap the number of retries or add jitter, every client hitting the failing dependency is now retrying, which multiplies load on a service that's already struggling, potentially preventing it from ever recovering — this is called a 'retry amplification' or 'metastable failure.' I'd add jitter, cap max retry attempts with a sane ceiling, and pair retries with a circuit breaker so that after enough consecutive failures, callers stop hammering the dependency entirely for a cooldown window instead of continuing to retry into a wall."

6.3 Circuit Breaker, Bulkhead, Rate Limiting, Backpressure

Circuit breaker: wraps a call to a dependency and tracks its failure rate. Three states: **Closed** (normal — calls go through), **Open** (failure threshold exceeded — calls fail immediately without even attempting the network call, giving the dependency breathing room to recover), **Half-Open** (after a cooldown, let a small number of test calls through to

check if the dependency has recovered; if they succeed, close the circuit; if not, re-open it). This prevents a struggling downstream service from also taking down every upstream caller that keeps waiting on its slow/failing responses.

Bulkhead: named after ship design — a ship is divided into watertight compartments so a hull breach in one doesn't sink the whole vessel. In software: isolate resources (thread pools, connection pools) *per dependency*, so a single slow/failing dependency can only exhaust *its own* allocated pool, not starve resources needed for calls to healthy dependencies. Without this, one bad downstream call can indirectly break unrelated features that share the same thread pool.

Rate limiting: protecting *your own* service from being overwhelmed by capping how many requests a client (or the system overall) can make in a given window. Algorithms (Chapter 13 goes deep): **token bucket** (tokens refill at a steady rate, each request consumes one, allows some burstiness up to the bucket size), **leaky bucket** (requests processed at a strictly constant rate, smoothing out bursts entirely), **fixed/sliding window counters** (simpler, but fixed windows have edge-of-window burst issues that sliding windows fix).

Backpressure: when a downstream consumer can't keep up with an upstream producer, backpressure is the mechanism for signaling "slow down" back up the chain, rather than silently dropping data or buffering unboundedly until memory runs out. In a queue-based system, this looks like: consumer lag grows → alert fires → either scale out consumers or the producer throttles itself. In a synchronous chain, it looks like: a service returns `429 Too Many Requests` or simply lets its connection queue fill up and stops accepting new connections, propagating the slowdown upstream instead of falling over.

6.4 Idempotency

Simple explanation: an operation is idempotent if performing it multiple times has the exact same effect as performing it once. `SET x = 5` is idempotent (run it 100 times, `x` is still 5). `x = x + 1` (increment) is **not** idempotent (run it 100 times, you get 100 different results).

Why it's the single most load-bearing concept in this whole chapter: retries are only safe if the thing being retried is idempotent. Every "what if the network fails right after the server processed the request but before the response arrives" scenario — which is unavoidable in distributed systems — resolves cleanly *only* if retrying is safe.

How to make a non-idempotent operation (like "charge $50") idempotent: the **idempotency key** pattern — the client generates a unique key (e.g., a UUID) per logical operation and sends it with the request. The server stores "I've seen this key, here was the result" before/while processing. If the same key arrives again (a genuine client retry after a lost response), the server returns the *stored result* instead of processing the

charge a second time. This is exactly how Stripe's and Razorpay's payment APIs work in practice, and it's the correct answer whenever an interviewer asks "how do you prevent a double charge on retry."

> **Interview question:** "A client calls 'create order,' the request succeeds, but the response is lost in the network. The client, seeing a timeout, retries. What happens?"
> **Ideal senior answer:** "Without an idempotency key, you get two orders — a real, damaging bug. With one: the client sends the same idempotency key on the retry (it should generate this key once per logical user action, before the first attempt, and reuse it on retries — not generate a new one each time, which would defeat the purpose). The server checks if it's already processed that key; if so, it returns the original result without creating a second order. I'd store the key with a TTL — long enough to cover realistic retry windows, short enough not to grow the table unboundedly — typically mapped to the actual order ID and status."

6.5 Distributed Locking, Leader Election, Consensus, Quorum

Distributed locking: ensuring only one process across multiple machines can do something at a time (e.g., only one worker should process a given job). Typically implemented via Redis (`SET key value NX PX ttl` — set-if-not-exists with an expiry, so a crashed lock-holder doesn't hold the lock forever) or via Zookeeper/etcd, which are purpose-built for this with stronger correctness guarantees than a plain Redis lock (the well-known "Redlock" debate — Redis's own lock recipe has known edge cases under certain failure/clock scenarios, and Zookeeper/etcd, built on a proper consensus protocol, are the more rigorously correct choice when the cost of a double-execution is genuinely severe, e.g., financial operations).

Leader election: the process by which a group of nodes agrees on which single node is "in charge" (e.g., which database replica accepts writes, which node coordinates a distributed job). Typically built on a consensus algorithm — Zookeeper (using ZAB) and etcd (using Raft) are the tools people actually reach for rather than implementing consensus themselves.

Consensus: the general problem of getting multiple nodes to agree on a single value, even when some nodes might fail or messages might be delayed — the theoretical foundation under leader election, distributed locks, and strongly-consistent distributed databases. **Raft** and **Paxos** are the two algorithms worth being able to name; you don't need to implement either, but you should know Raft is the one most modern systems (etcd, Consul, CockroachDB) use because it's explicitly designed to be more understandable than Paxos while providing equivalent guarantees.

Quorum: a voting-based approach to reads/writes in a leaderless replicated system (Cassandra, DynamoDB-style). Define **N** (total replicas), **W** (replicas that must acknowledge a write to consider it successful), **R** (replicas that must respond to a read). The rule **W + R > N** guarantees every read overlaps with at least one replica that has the latest write — a strong-consistency guarantee achieved *without* a fixed leader. Common configurations: `N=3, W=2, R=2` (balances consistency and availability — tolerates one node being down for either reads or writes) versus `N=3, W=1, R=1` (fast, but no consistency guarantee — you might read stale data) versus `N=3, W=3, R=1` (strong write consistency at the cost of write availability — all 3 must be up to write).

> **Interview question:** "You're using a Cassandra-style database with N=3. How would you configure W and R for a system where staleness is unacceptable but you still want to tolerate one node being down?"
> **Ideal senior answer:** "W=2, R=2 — since W+R=4 > N=3, every read is guaranteed to see the most recent write. This tolerates any single node being unreachable for both reads and writes, since 2 out of 3 remaining nodes can still satisfy the quorum. Going to W=3 would make writes fail entirely if even one node is down, which is usually too strong a trade for the marginal consistency gain over W=2/R=2."

6.6 Ordering, Duplicates, and Delivery Semantics

Ordering: in a distributed system, messages/events from different producers (or even the same producer, if retried) can arrive out of order at a consumer. Where order matters (e.g., "item added to cart" must be processed before "item removed from cart" for the same cart), you need an explicit ordering mechanism — typically a partition key that routes all events for the same logical entity to the same ordered stream/partition (this is exactly why Kafka partitions by key — Chapter 7).

Duplicate messages: the network can deliver the same message more than once (a producer retries after not receiving an ack, even though the message *did* arrive) — this is unavoidable in any at-least-once delivery system, so consumers need to handle duplicates gracefully, usually via the same idempotency-key pattern from Section 6.4, applied at the message-consumption layer.

The three delivery semantics — a guaranteed interview question:

Semantic	Guarantee	Trade-off
At-most-once	Message delivered zero or one times — never duplicated	Simple, fast, but **can silently lose messages** — acceptable only for data where loss is tolerable (some metrics, some logs)
At-least-once		The most common real-world default (Kafka, SQS standard). Requires idempotent consumers to handle the duplicates safely

Semantic	Guarantee	Trade-off
	Message delivered one or more times — never lost	
Exactly-once	Message delivered and processed exactly once, no loss, no duplication	The hardest and most expensive guarantee — usually achieved by combining at-least-once delivery *with* idempotent processing (which, done right, is functionally equivalent to exactly-once from the consumer's perspective) rather than true exactly-once delivery at the transport layer, which is extremely difficult across independent systems

> **Interview question:** "Kafka advertises 'exactly-once semantics.' How is that actually achieved?"
> **Ideal senior answer:** "Kafka's exactly-once is really at-least-once delivery plus idempotent producers (each producer gets a sequence number per partition, so the broker can detect and drop a duplicate retry) plus transactional writes across topics/partitions for consume-process-produce pipelines. It's scoped to Kafka-to-Kafka pipelines — the moment you're writing a side effect to an external system (calling a payment API, writing to a different database), you're back to needing the idempotency-key pattern at that boundary, because Kafka's guarantees don't extend outside Kafka itself."

6.7 Clock Problems

Why clocks are a distributed-systems problem at all: each machine has its own physical clock, and clocks on different machines drift — even with NTP synchronization, you can have milliseconds to seconds of skew. This breaks any logic that assumes "later timestamp = happened later," which is a genuinely dangerous assumption to bake into distributed logic (e.g., "last-write-wins" conflict resolution using wall-clock timestamps can silently pick the *wrong* write if one node's clock is fast).

Logical clocks (Lamport timestamps, vector clocks) solve this by tracking causality — "did event A happen before event B" — without relying on wall-clock time at all, just on the order of message exchanges between nodes. You don't need to implement these, but knowing they exist and *why* (wall clocks lie; causality doesn't) is the level of depth interviewers are checking for.

Google Spanner's TrueTime is worth namedropping if you want to show depth: Spanner uses GPS and atomic clocks across its datacenters to bound clock uncertainty to a known small window, and *waits out* that uncertainty window before committing a transaction — turning "we can't trust clocks" into "we know exactly how much we can't trust them, and

we design around that number." It's the exception that makes CP-with-good-latency achievable at global scale, precisely because it treats clock uncertainty as a first-class, measured quantity instead of ignoring it.

6.8 Distributed Transactions: 2PC, Saga, Outbox, CDC, CQRS, Event Sourcing

This is the section that most directly matters for microservices and fintech interviews — "how do you keep data consistent across services that each own their own database" is one of the highest-signal questions you'll get.

Two-Phase Commit (2PC)

A coordinator asks all participants "can you commit?" (**prepare phase**); if all say yes, it tells them all to actually commit (**commit phase**); if any says no, it tells them all to abort. This gives strong atomicity across multiple databases — but at a real cost: it's **blocking** (if the coordinator crashes between phases, participants are stuck holding locks indefinitely, waiting), and every participant must be available and fast, which fights directly against availability and service independence. **In practice, 2PC is rarely used across microservices** in modern systems — it's mentioned mostly so you can correctly explain *why* people avoid it in favor of Sagas.

Saga Pattern

A sequence of local transactions, one per service, where each step publishes an event/message that triggers the next step — and critically, **each step has a corresponding compensating action** to undo it if a later step fails. Example: "Book a trip" = reserve flight → reserve hotel → charge payment. If charging payment fails, you run compensating actions in reverse: cancel hotel reservation, cancel flight reservation.

- **Choreography-based Saga:** each service listens for events and reacts, publishing its own events in turn — no central coordinator. Simpler to start, but the overall flow becomes implicit, spread across services, and harder to trace/debug as steps grow.
- **Orchestration-based Saga:** a central orchestrator service explicitly calls each step in sequence and handles failures/compensation. More visible and easier to reason about and debug, at the cost of the orchestrator being a more central, more critical component.

> **Interview question:** "Design the transaction flow for an e-commerce checkout that reserves inventory, charges payment, and creates a shipment — across three separate services."
> **Ideal senior answer:** "I'd use an orchestrated Saga rather than 2PC — 2PC's blocking behavior and availability coupling don't fit a system where these are genuinely independent services with independent uptime. An orchestrator calls: reserve inventory → charge payment → create shipment, in order. If payment fails, compensate by releasing the inventory reservation. If shipment creation fails, compensate by refunding the payment and releasing inventory. Each compensating action needs to itself be idempotent and retryable, since the failure-handling path is just as much a distributed operation as the happy path. I'd choose orchestration over choreography here specifically because a financial flow benefits from having one obvious place to look to understand the whole transaction's state, rather than tracing it across three services' independent event handlers."

Outbox Pattern

The problem it solves: a service needs to both (a) update its own database and (b) publish an event about that update (e.g., to Kafka) — and it needs *both to happen, or neither*. If you write to the DB and then publish the event as two separate operations, a crash between them leaves you inconsistent (DB updated, no event published — or vice versa if you publish first).

The solution: write the event into an `outbox` table in the *same database transaction* as the actual business data change — so both commit atomically, using the guarantee your database already gives you for free. A separate background process (or CDC, below) reads new rows from the outbox table and publishes them to the message broker, marking them as sent. This turns a hard distributed-consistency problem into a single-database-transaction problem, which is a solved problem.

CDC (Change Data Capture)

Reading a database's internal replication/transaction log (e.g., MySQL's binlog, Postgres's WAL) to capture every row-level change as a stream of events — without the application having to explicitly publish anything. **Debezium** is the standard open-source tool for this. CDC is commonly used *to implement* the Outbox pattern's publishing step (tail the outbox table via CDC instead of a custom polling worker), and more broadly to sync data into other systems (a search index, a data warehouse, a cache) without coupling the source application to every downstream consumer.

CQRS (Command Query Responsibility Segregation)

Splitting the **write model** (optimized for handling commands/writes — validation, business rules, normalized schema) from the **read model** (optimized for queries — denormalized, possibly a completely different database technology, built specifically for how reads actually query the data). Writes go through the command side; the read side is updated asynchronously (often via the events the write side emits) and can be shaped however read performance demands, even duplicated into multiple specialized read models for different query patterns.

When it's worth the complexity: when read and write workloads have genuinely different scaling needs or query shapes — e.g., writes are simple single-entity updates, but reads need complex aggregated views across many entities (an analytics dashboard, a search page). **When it's not:** most CRUD services, where the extra moving parts (sync lag between write and read models, more infrastructure) aren't earning their cost — this is a frequently over-applied pattern, and knowing when *not* to use it is itself a strong signal.

Event Sourcing

Instead of storing the *current state* of an entity, you store the full sequence of *events* that led to that state (e.g., not "balance = $150" but "deposited $100," "deposited $75," "withdrew $25" — replaying these gives you $150). Current state is derived by replaying events, often with periodic **snapshots** so you don't replay from the beginning every time.

- *Advantages:* a complete audit trail for free (extremely valuable in fintech — Chapter 20), the ability to reconstruct state as of any point in time, and natural fit with CQRS (the event stream *is* the write model; read models are projections of it).
- *Disadvantages:* real complexity — querying "current state" isn't a simple lookup anymore, schema evolution of events over time is genuinely tricky, and it's a significant mental model shift for a team.
- *When to use:* domains where the *history* of changes is itself valuable or required (ledgers, audit-heavy fintech flows, collaborative editing). *When not to:* simple CRUD where nobody will ever ask "what was this record's state on March 3rd" and an audit log table would suffice more simply.

Chapter 6 Interview Drill

1. Explain, precisely, why you can't distinguish a slow node from a dead one — and why that fact drives timeouts, retries, and circuit breakers.
2. Walk through the idempotency-key pattern end to end for a "create order" API.
3. State the quorum formula and configure N/W/R for a system that must tolerate one node failure while guaranteeing read-after-write consistency.
4. Explain why 2PC is avoided across microservices, and what pattern replaces it.

5. Explain the Outbox pattern — what specific failure mode does it prevent that naively publishing an event after a DB write does not?
6. When would you reach for CQRS, and when would you deliberately avoid it?

Next → Chapter 7: Messaging & Event-Driven Architecture — Kafka internals, RabbitMQ, SQS/SNS, and how to choose between them.